Università
Ca'Foscari
Venezia

Constraint Satisfaction

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Games Vs. Planning Problems

In many optimization problems, the path to the goal is irrelevant

- the goal state itself is the solution

Find configuration satisfying constraints

- State space = set of "complete" configurations

Constraint satisfaction problems (CSPs) arise in many application areas: they can be used to formulate

- scheduling tasks
- robot planning tasks
- Puzzles
- molecular structures
- sensory interpretation tasks
- ...

In such cases, we can use search algorithms to keep a single "current" state and try to improve it

Variables, Domains, Constraints

We formalize constraint satisfaction problems in terms of:

Variables entities that can have multiple values assigned to them. The set of assignments of values to variables constitues the search space

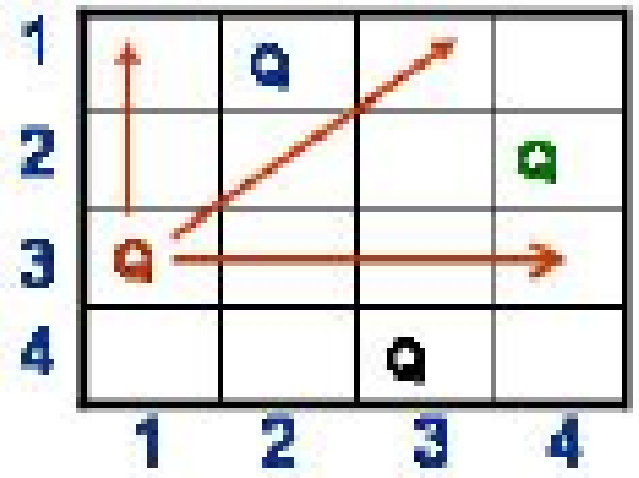
Domains the set of values that can be assigned to a variable

Constraints conditions on the valid assignments of values to variables, in the context of the other assignments.

Example: n-queens

Place n queens on an $n \times n$ chessboard so that none can attack any other

- no two queens on the same row, column, or diagonal

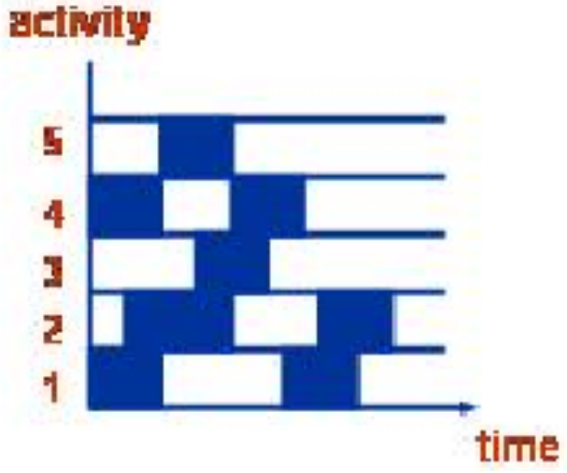


Variables	board positions in n×n chessboard
Domains	Q(ueen) or B(lank)
Constraints	Two positions on a line (vertical, horizontal, diagonal) cannot both be Q

Università
Ca'Foscari
Venezia

Example: Scheduling

Choose time for activities, *e.g.*, observation on James Webb telescope, or tterms to take required classes



Variables activites

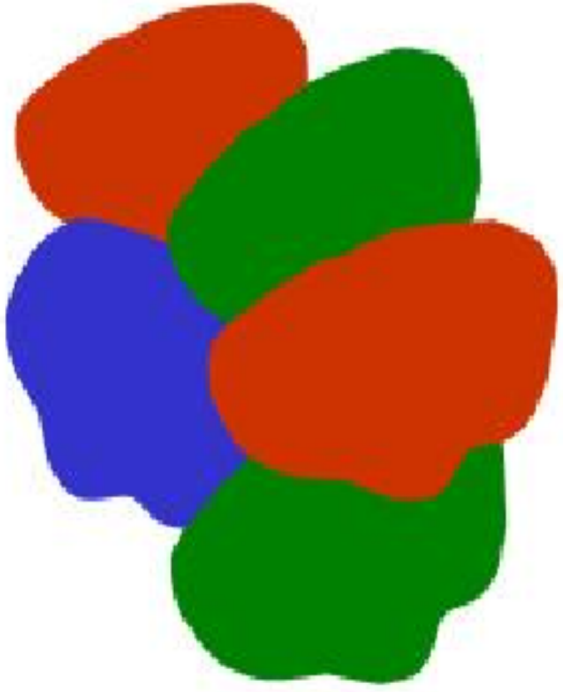
Domains set of start times (or "chunks"of time)

Constraints

1. Activities that use same resource cannot overlap in time
2. Preconditions satisfied

Example: Graph Coloring

Pick colors for map regions, avoiding coloring adjacent regions with the same color



Variables regions

Domains colors allowed

Constraints adjacent regions mut have different colors

3-SAT

The original NP-complete problem

Find values for boolean variable $A, B, C, \ldots$ that satisfy the formula

$$(A \vee B \vee \neg C) \wedge (\neg A \vee C \vee B)$$

Variables	clauses
Domains	boolean variable assignments
Constraints	clauses with shared boolean variable must agree on value of variable

Solving CSP

Good News very general & interesting class of problems

Bad News includes intractable problems

Good behavior is a function of domain not the formulation as CSP

Since CSP problem can be NP-Hard one cannot hope to find a (global) optimal solution, but has to revert to local approaches

Solving CSP involves some combination of

- **Constraint propagation** to eliminate values that cannot be part of the solution
- **Search** to explore valid assignments

Constraint Propagation (Arc Consistecy)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution

$$v_i \rightarrow v_j$$

Directed arc (v_i, v_j) is arc consistent if $\forall x \in D_i, \exists y \in D_j$ such that (x, y) is allowed by the constraint on the arcWe can achieve

consistency on arc by **deleting values** from D_i (domain of variable at tail of constraint arc) that fails this condition

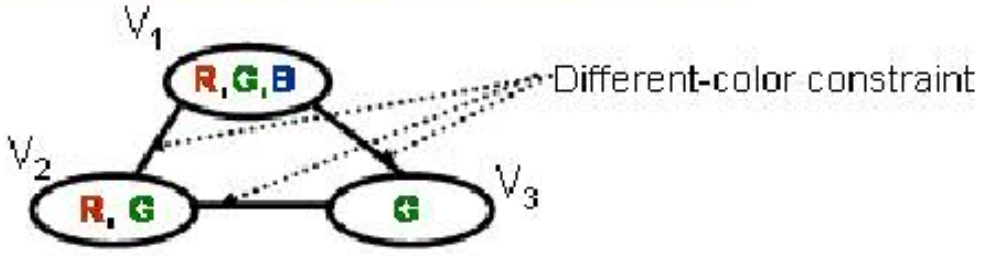
Assume domains are size at most d and there are e binary constraints:

- a simple algorithm for arc consistency is $O(ed^3)$
- just verifying arc consistency takes $O(d^2)$ for each arc

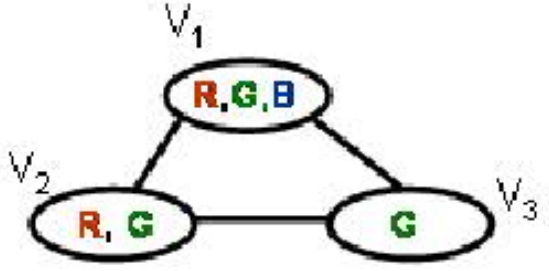
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



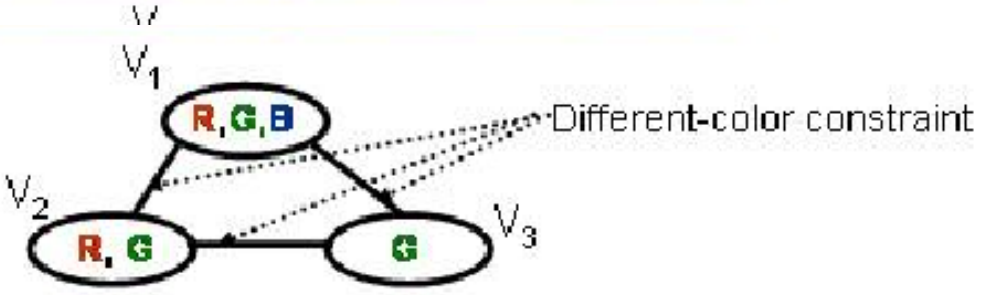
Arc examined	Value deleted



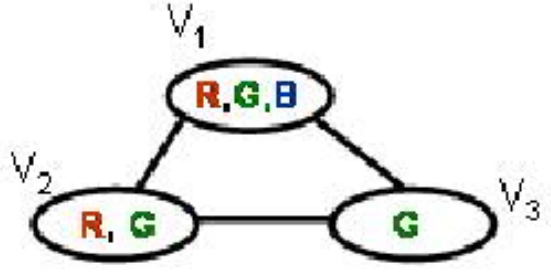
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



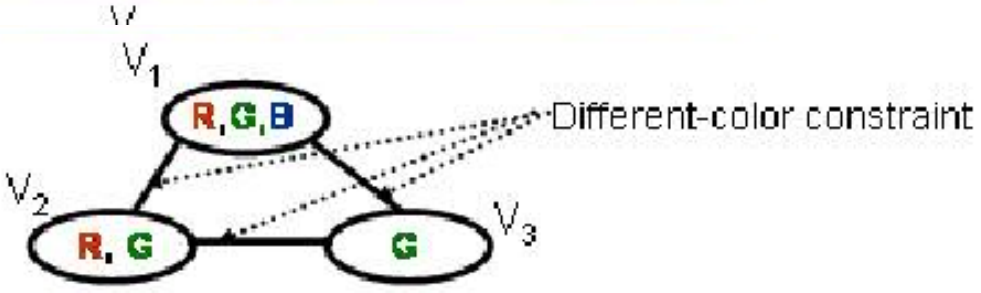
Arc examined	Value deleted
$V_1 - V_2$	none



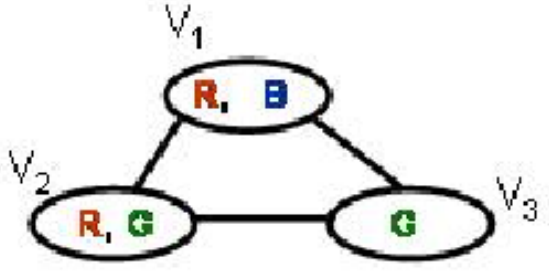
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



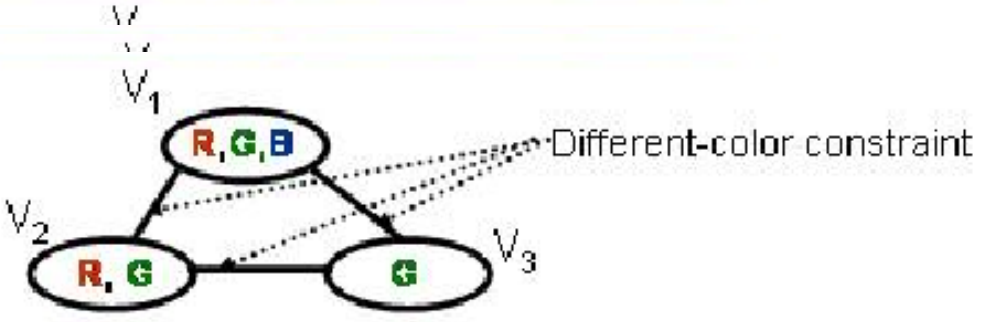
Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)



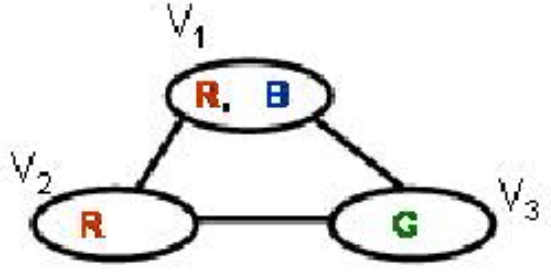
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



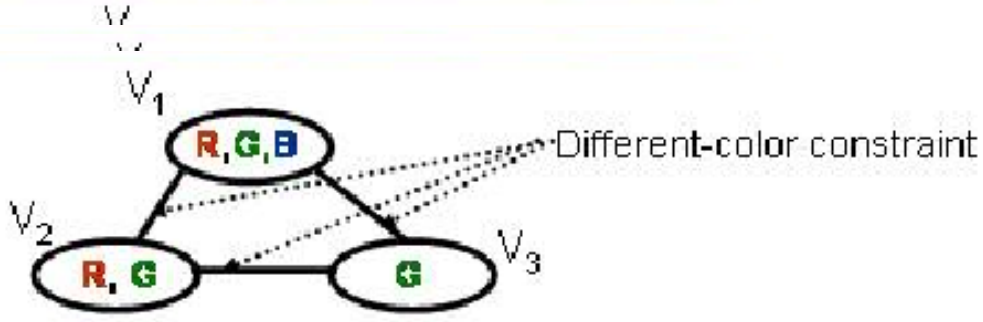
Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)
$V_2 - V_3$	V_2(G)



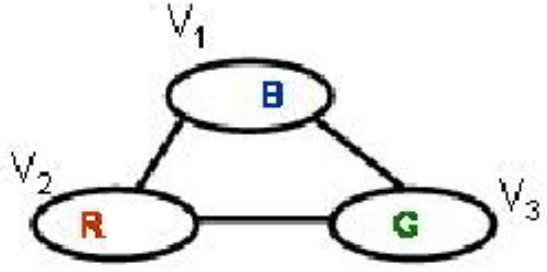
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



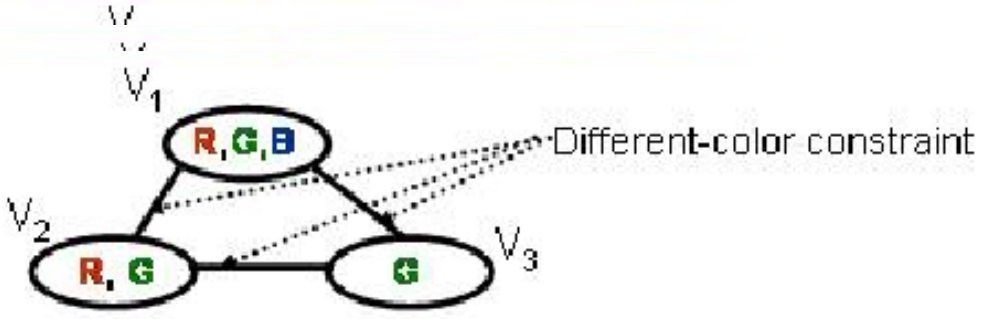
Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)
$V_2 - V_3$	V_2(G)
$V_1 - V_2$	V_1(R)



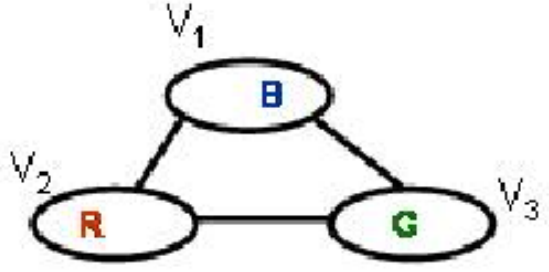
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated



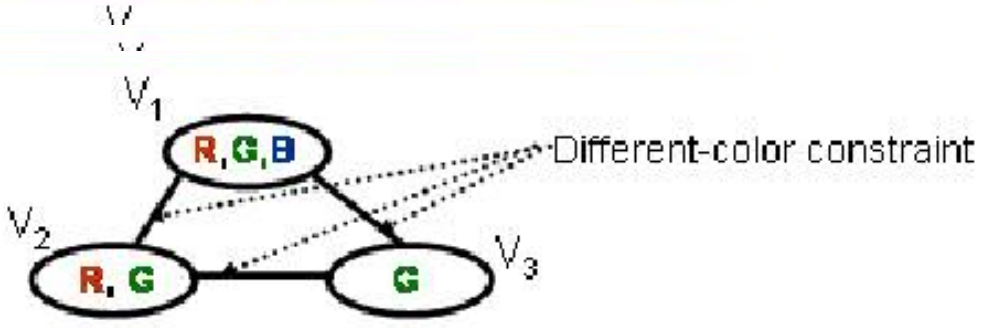
Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)
$V_2 - V_3$	V_2(G)
$V_1 - V_2$	V_1(R)
$V_1 - V_3$	none



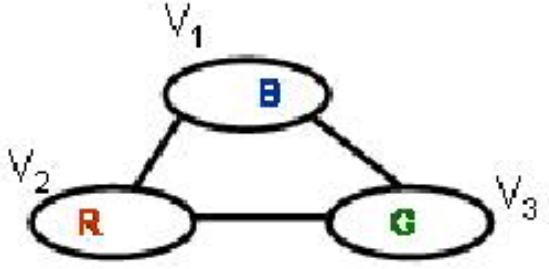
Example: Graph Coloring

Graph Coloring

Initial Domains are indicated

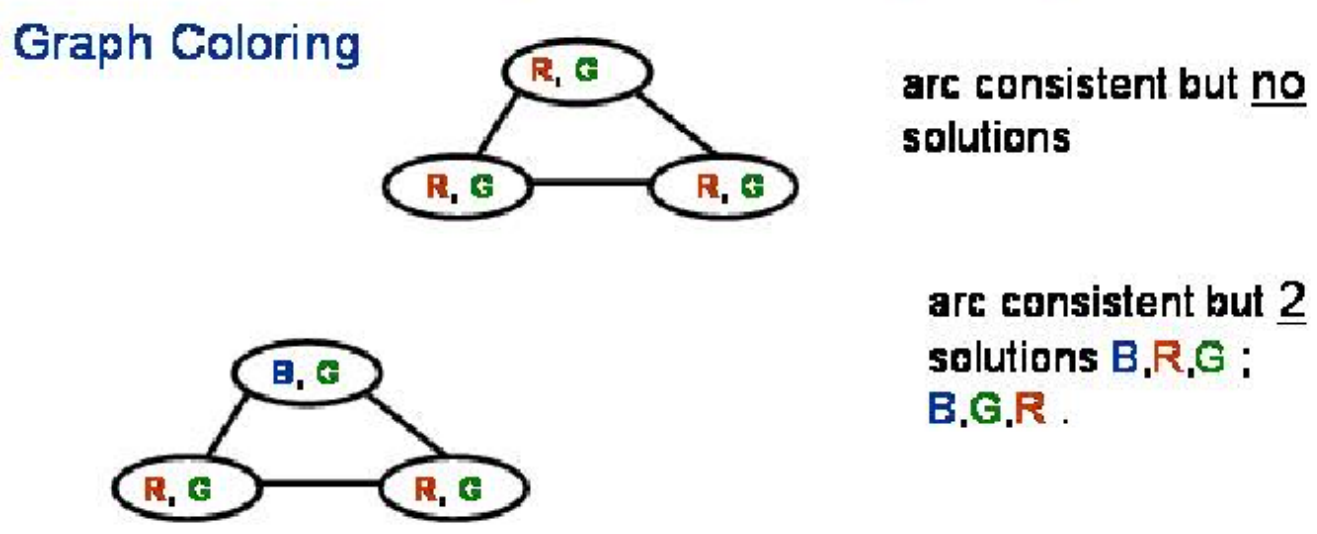


Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)
$V_2 - V_3$	V_2(G)
$V_1 - V_2$	V_1(R)
$V_1 - V_3$	none
$V_2 - V_3$	none



Arc Consistency is not enough!

There are situations where constraint propagation does not guarantee a solution

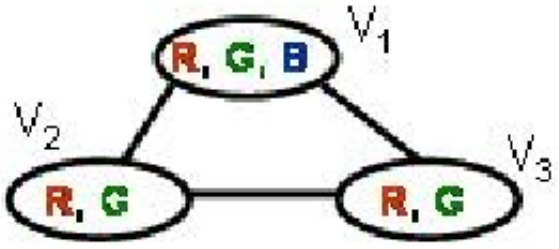


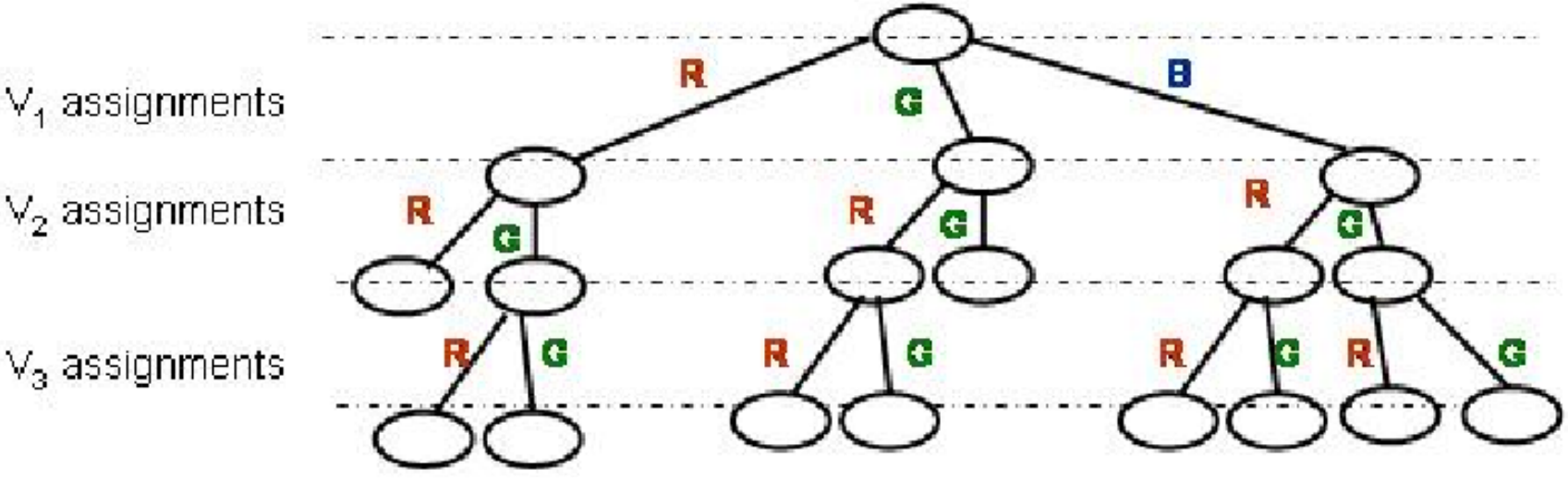
In general, when we have too many values in domain and/or constraints are weak arc consistency doesn't do much

In these situation we have to search the state-space for solutions

Backtracking

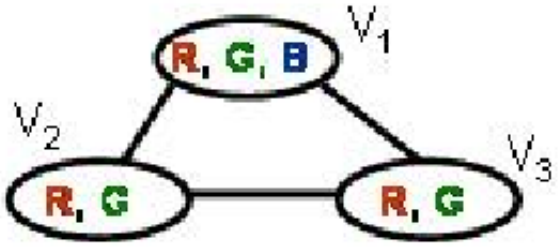
Simplest approach is pure backtracking (depth-first search)

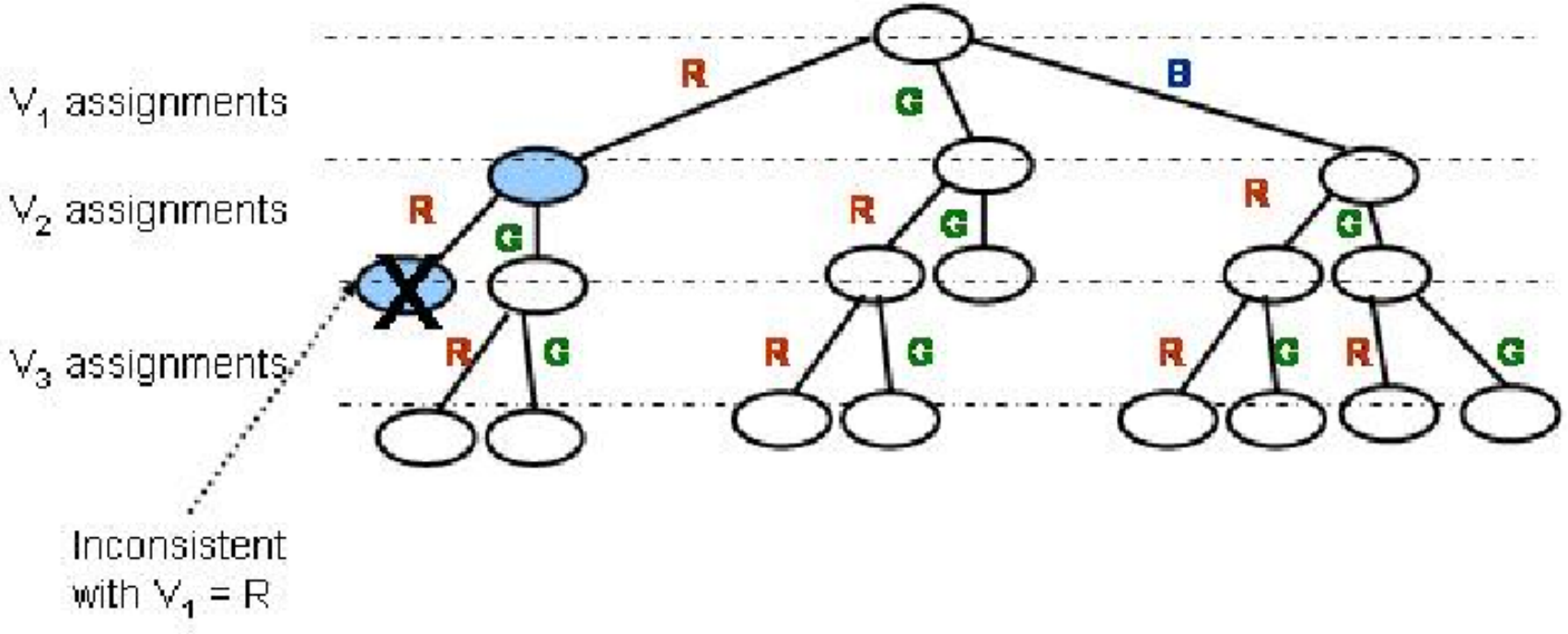




Backtracking

Simplest approach is pure backtracking (depth-first search)

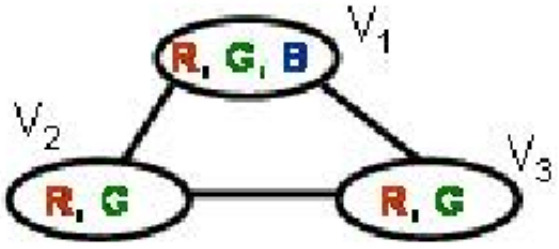


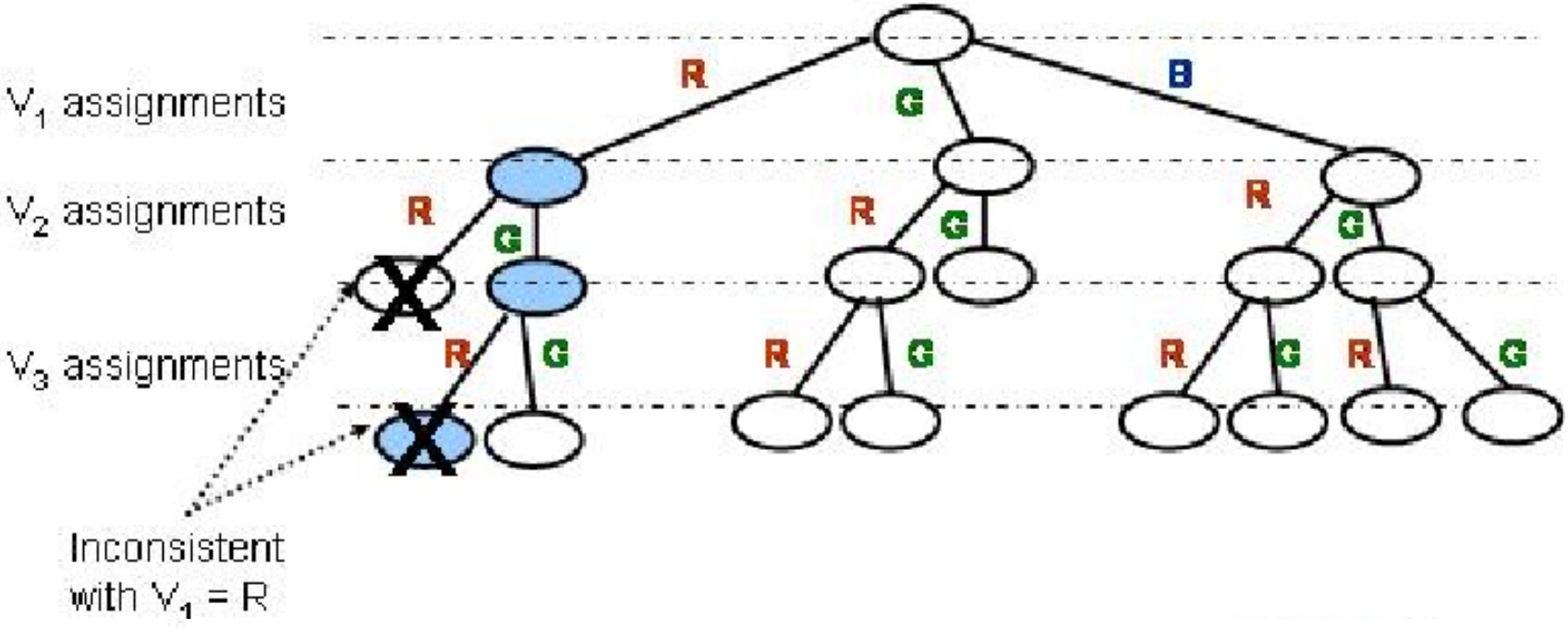


Università
Ca'Foscari
Venezia

Backtracking

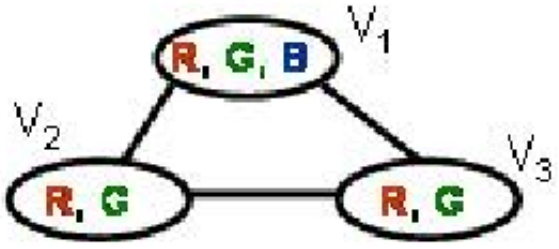
Simplest approach is pure backtracking (depth-first search)

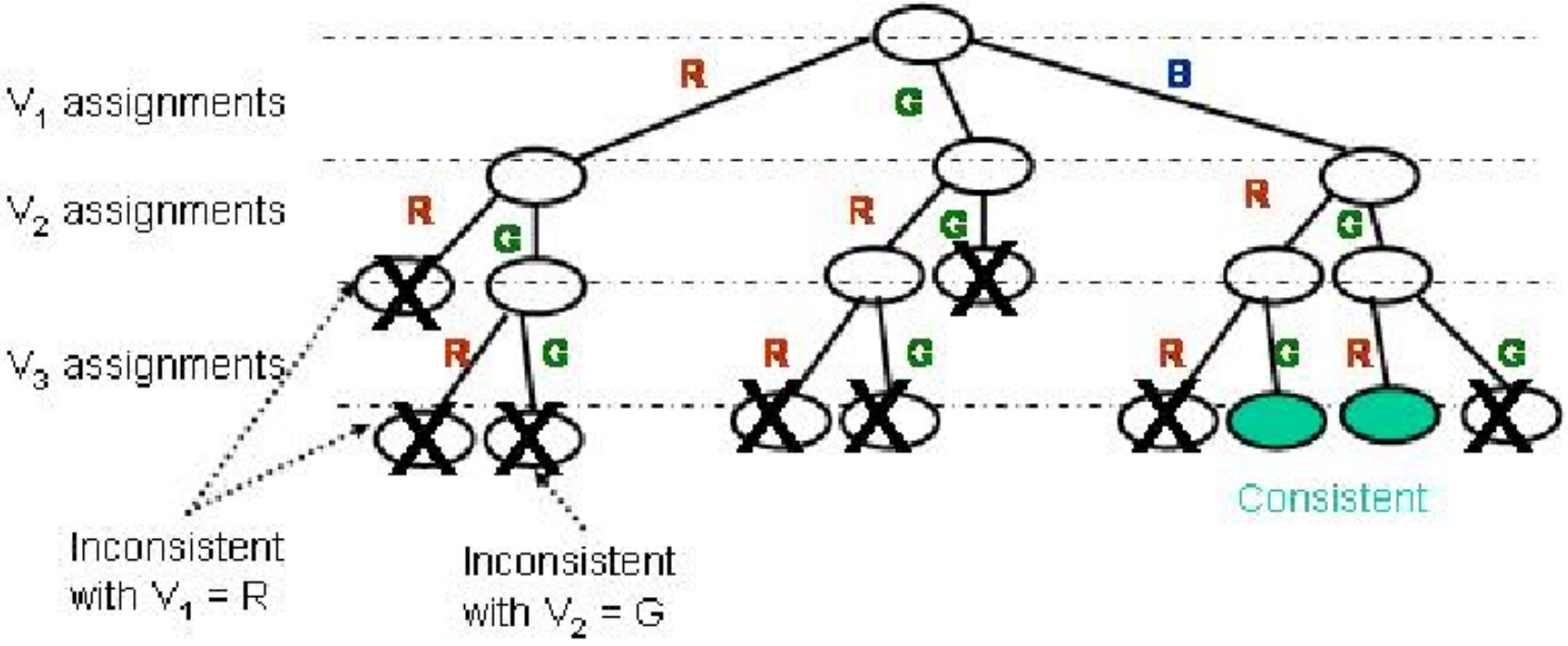




Backtracking

Simplest approach is pure backtracking (depth-first search)





Combine Backtracking & Constraint Propagation

A node in BT tree is a partial assignment in which some variable are set to a (tentative) value

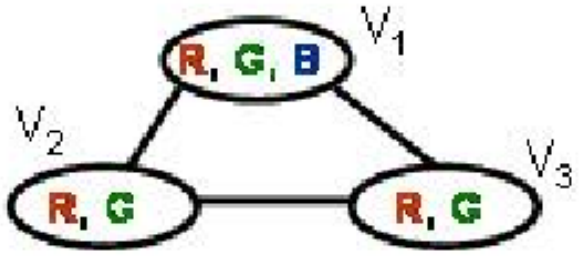
Use constraint propagation to propagate effect of this tentative assignment

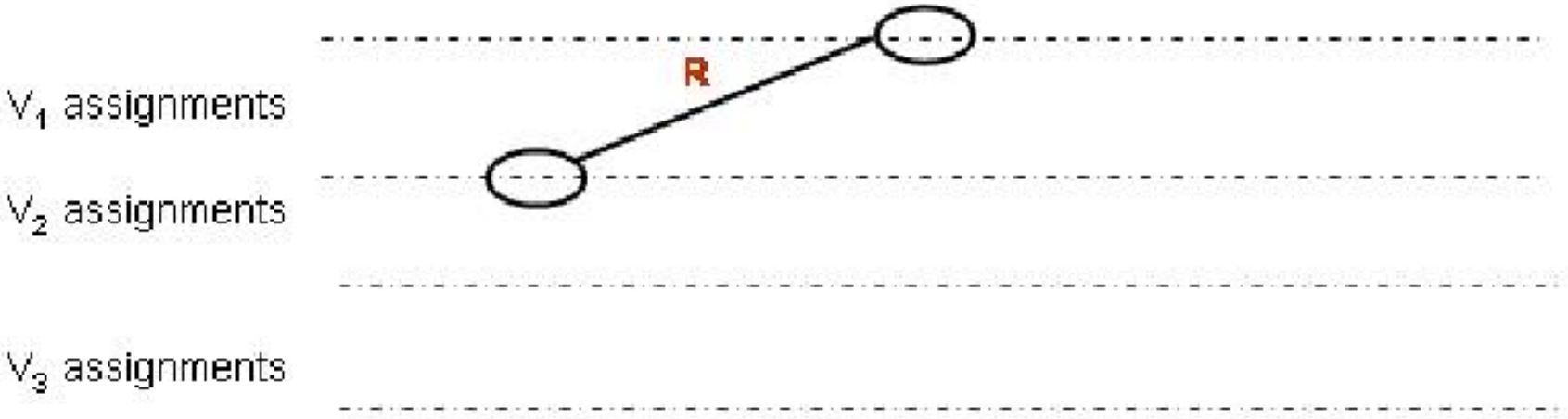
- Eliminate states inconsistent with current hypothesis

How much propagation to do?

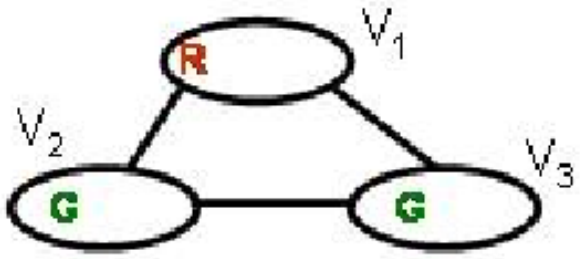
Forward checking: Do just local propagation from domains with unique assignments

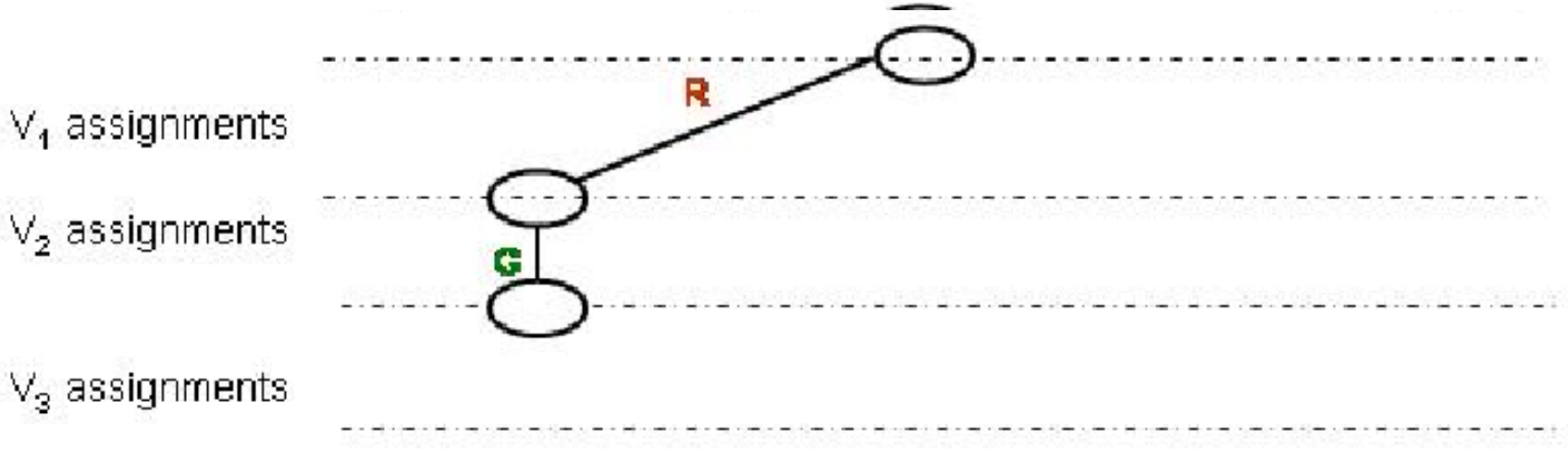
Backtracking & Forward Checking



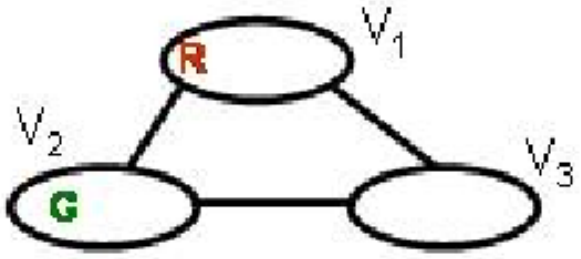


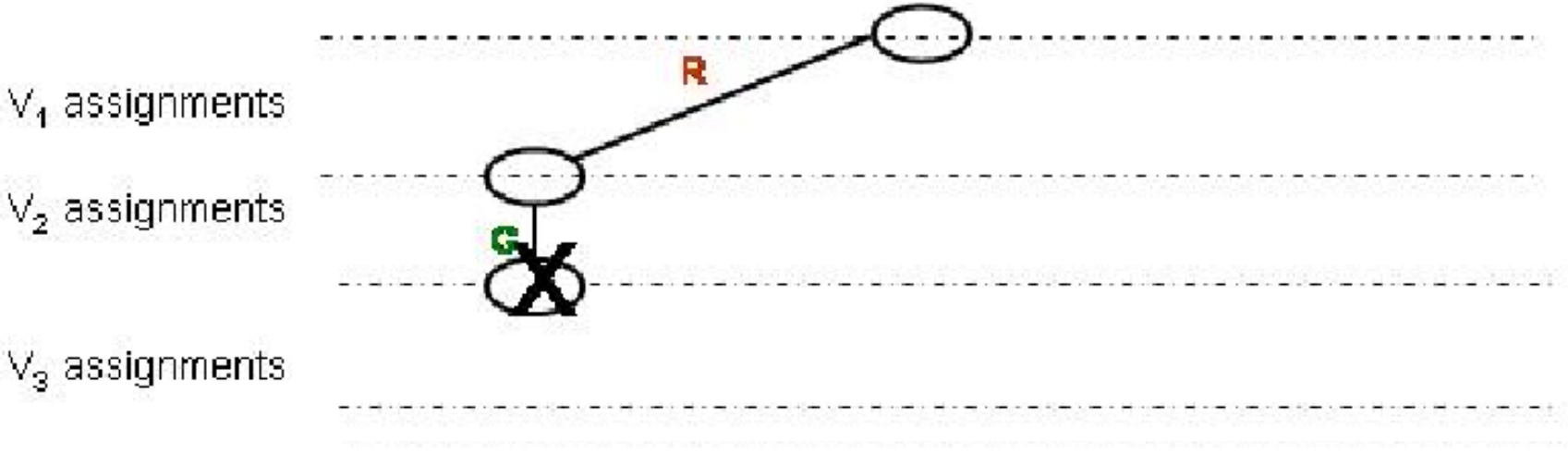
Backtracking & Forward Checking



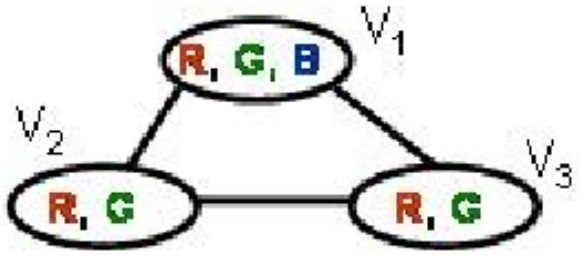


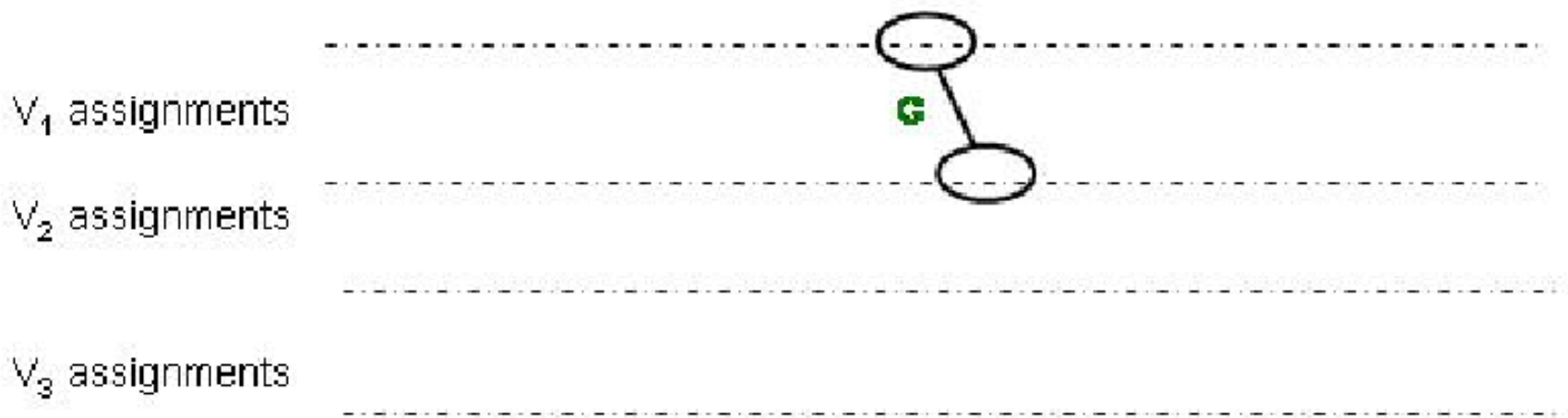
Backtracking & Forward Checking



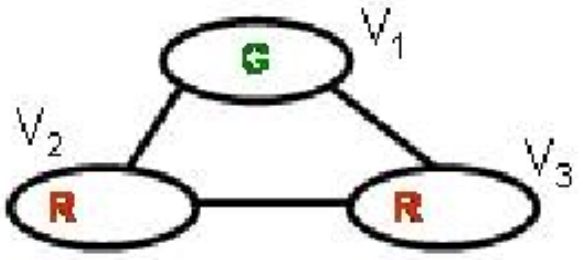


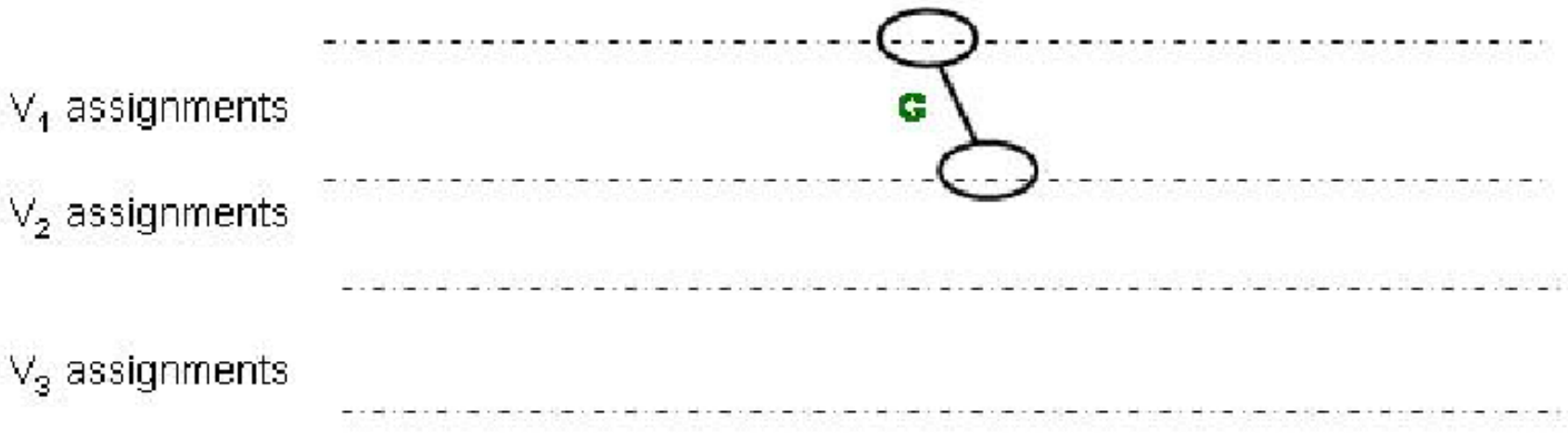
Backtracking & Forward Checking



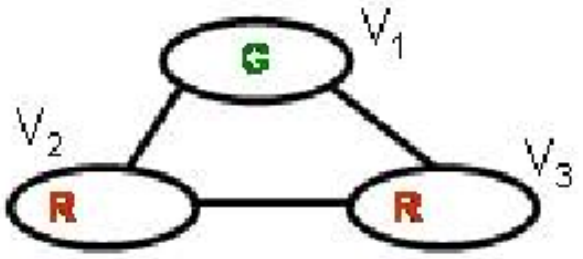


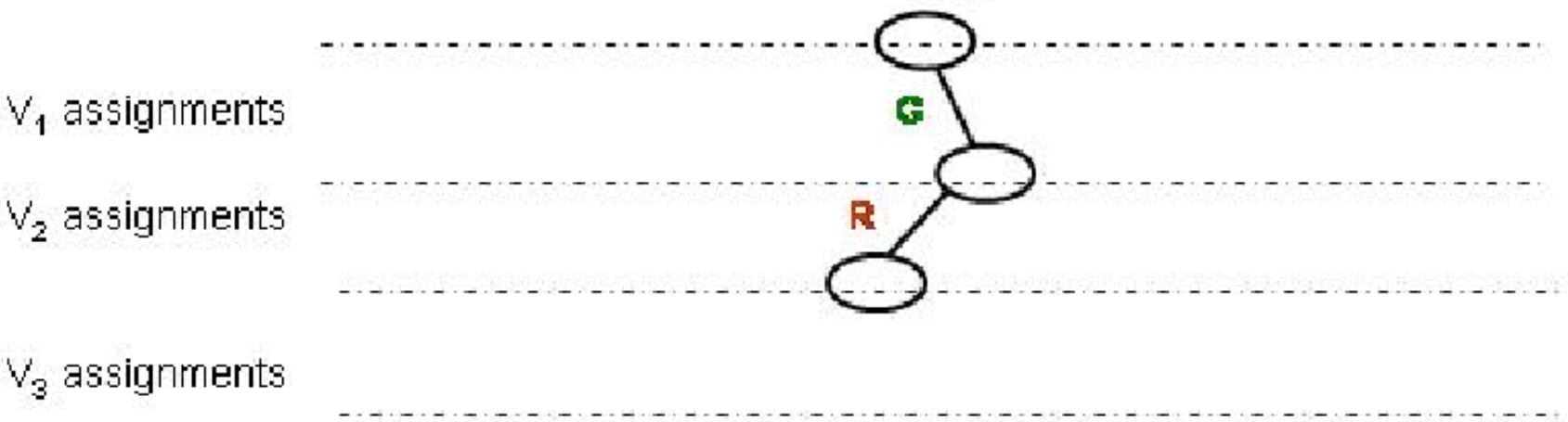
Backtracking & Forward Checking



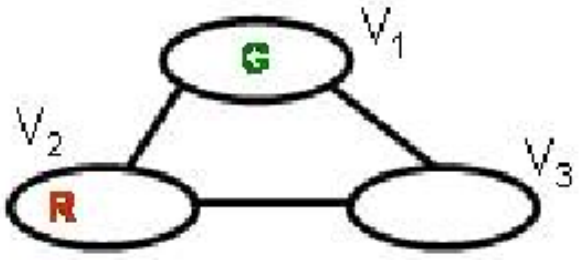


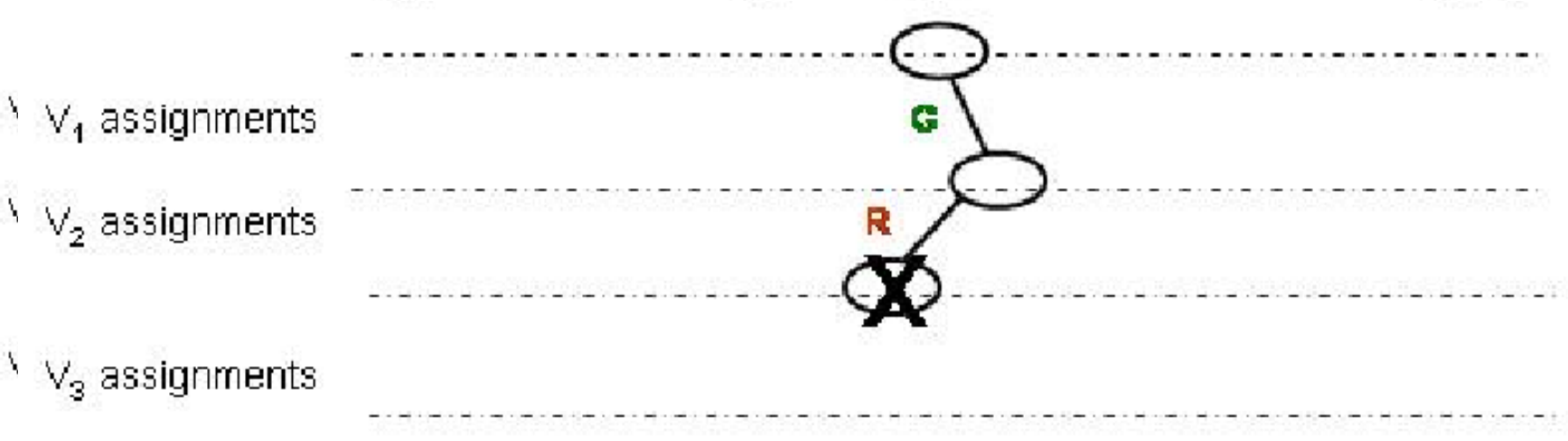
Backtracking & Forward Checking



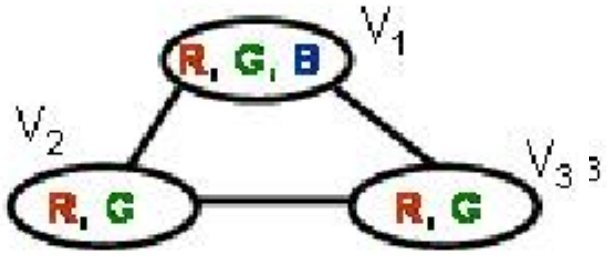


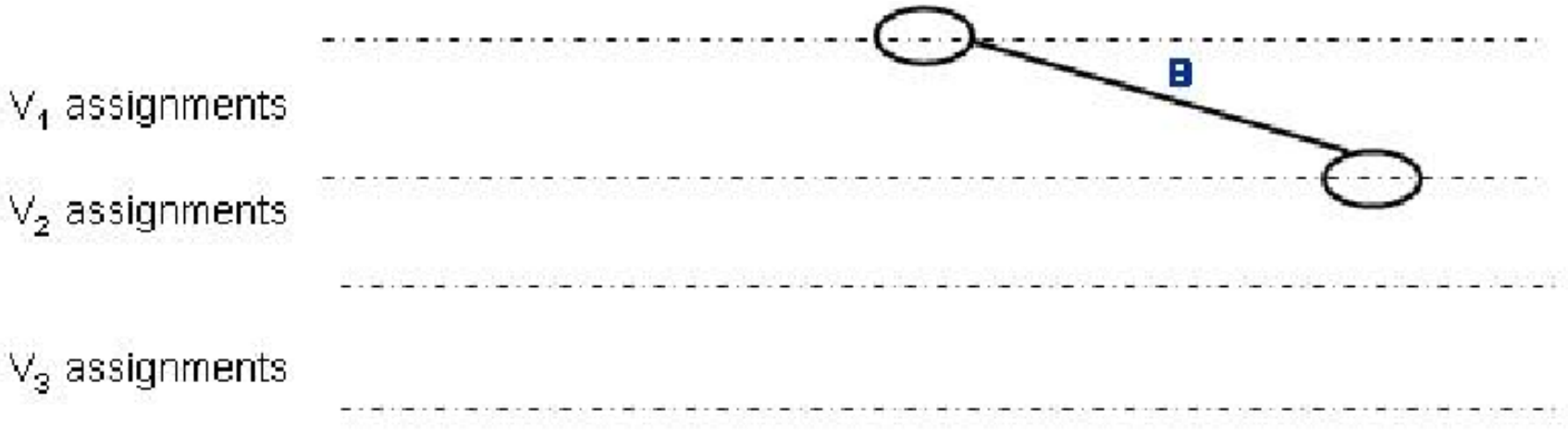
Backtracking & Forward Checking



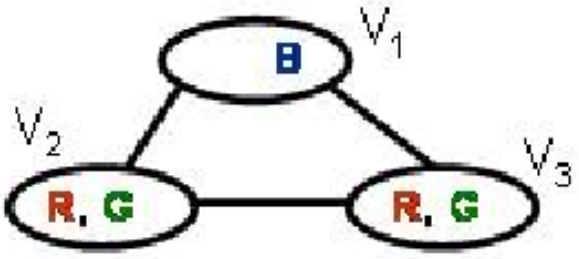


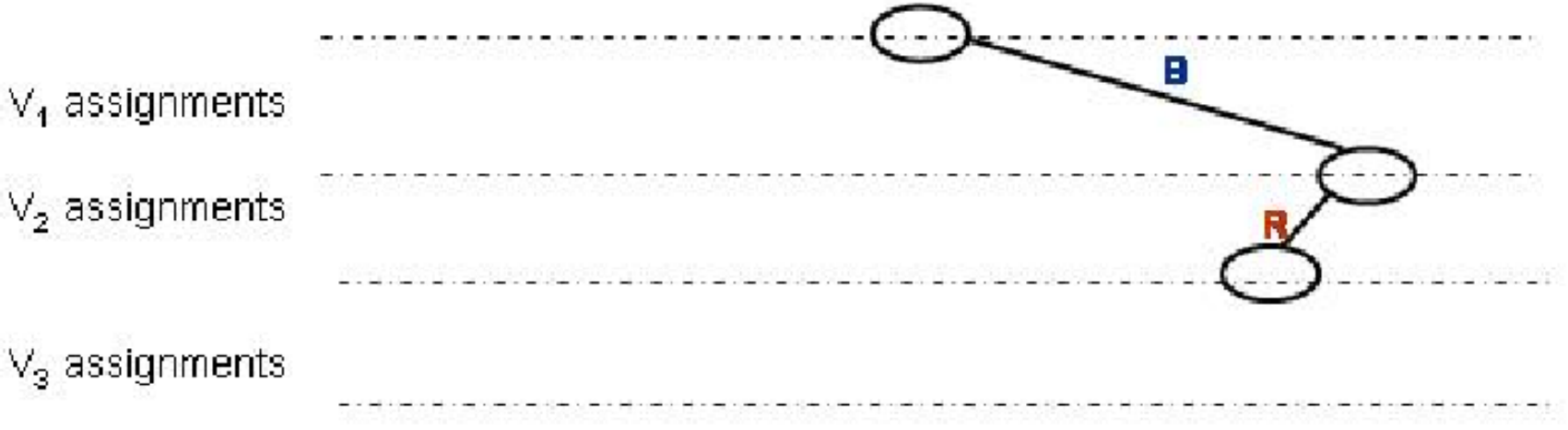
Backtracking & Forward Checking



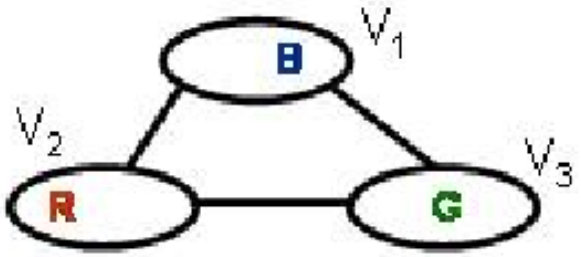


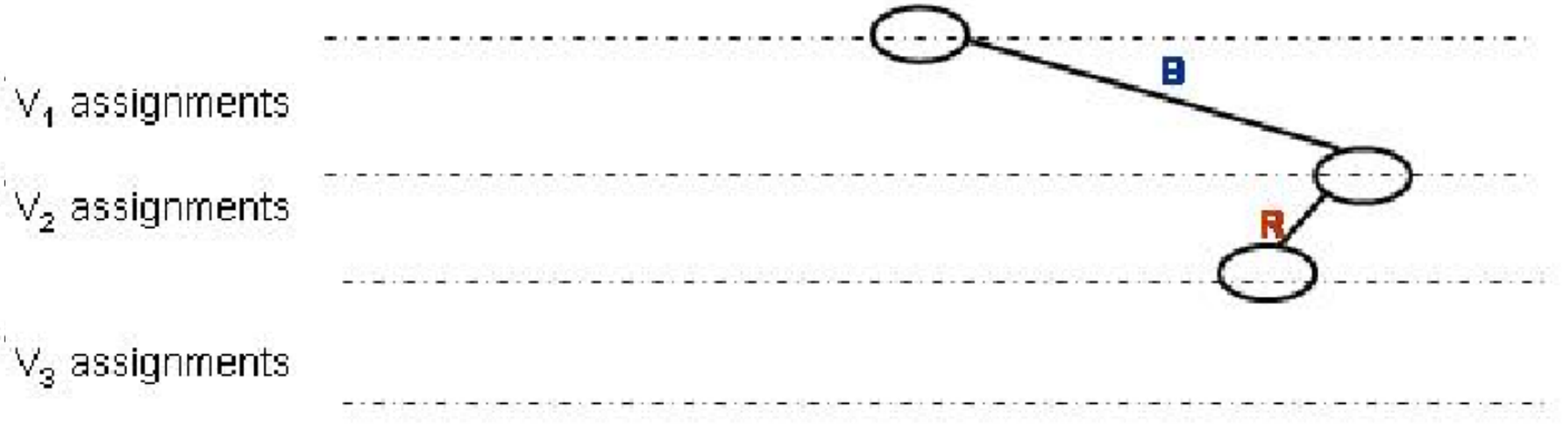
Backtracking & Forward Checking





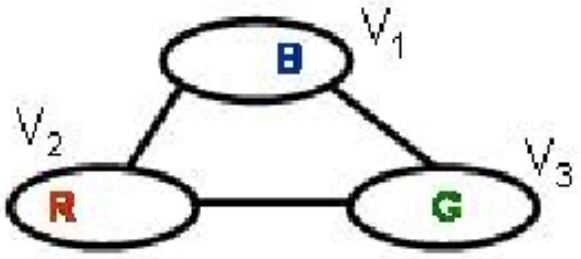
Backtracking & Forward Checking

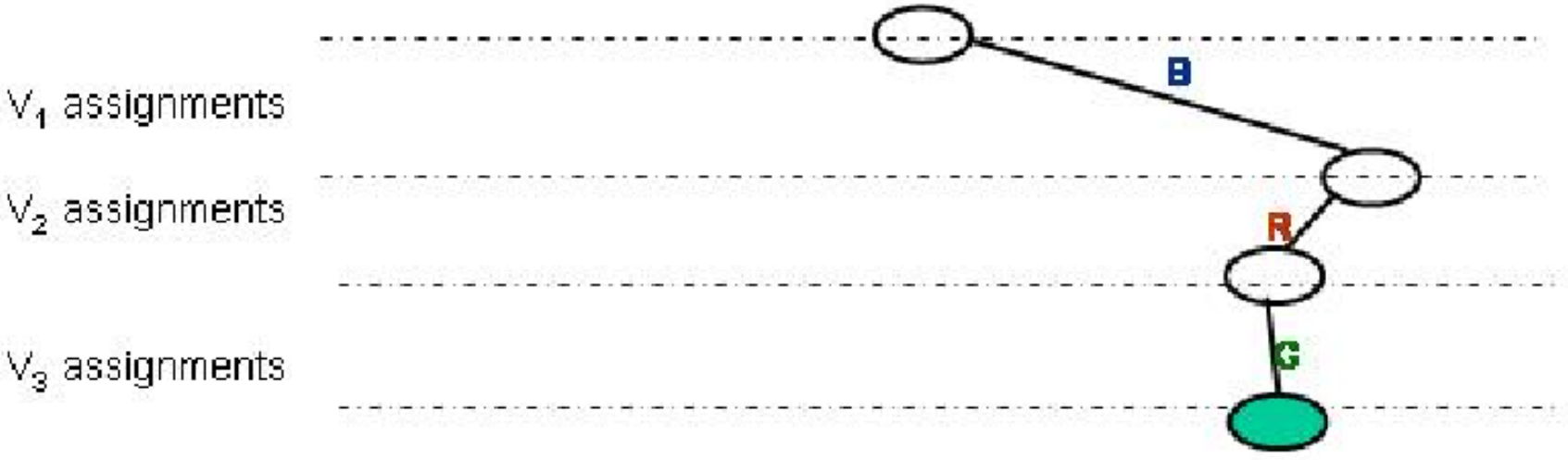




Università
Ca'Foscari
Venezia

Backtracking & Forward Checking





Dynamic ordering

Traditional backtracking uses fixed ordering of variables & values

You can usually do better by choosing an order as the search proceeds

Most Constrained Variable: Pick variable with fewest legal values to assign next (minimizes branching factor)

Least Constrained Variable: Pick value that rules out the fewest values from neighboring domains